Lingnan CrownCred

group name: Lucky Degen

group members: LIU Yifei 3162122

YIN Nanzhu 5597371

ZHANG Yaxuan 5545160

XU Xinghao 5597797

ZHOU Haoxiang 5548320

YUAN Zhuoer 5505275

Github url: https://github.com/Chaosrussell/Lingnan-CrownCred

Youtube url: https://youtu.be/

abstract: This project implements a blockchain-based academic incentive prototype system, Lingnan CrownCred (EduReward). The system employs a hybrid architecture of "off-chain grades + on-chain rewards": administrators input student grades on the front end and store them in local JSON/browser storage, which the front end uses to determine reward eligibility; when a student meets the A grade requirements, the student independently calls the smart contract via MetaMask to claim the ERC-20 token reward and mint an ERC-721 NFT. The on-chain portion is only responsible for reward distribution, claim status verification, and event logging, thereby reducing gas costs and operational complexity while maintaining the transparency and immutability of rewards. We conducted a Slither static audit and functional testing of the contract and completed deployment and demonstration on the Sepolia testnet.

key words: Blockchain; Educational incentives; Off-chain performance; ERC-20; ERC-721

Table of contents

1. Introduction and Problem Analysis 3
1.1 Technical Background of Academic Incentives 3
1.2 Technical Adaptability of Blockchain 3
1.3 Project Technical Objectives (Lingnan CrownCred) 3
2. System Design & Architecture 4
2.1 High-Level System Overview 4
2.2 Core Workflow Design 5
2.3 Key Mechanism Design 6
3. Smart Contract Layer 6
3.1 Design and Implementation of ERC-20 Token LNTC 7
3.2 Implementation and Metadata Storage of Achievement-Based NFTs (ERC-721) 7
3.3 Automated Reward Distribution Contract 8
3.4 Interaction methods between the front-end and smart contracts 10
3.5 Smart Contract Layer Summary 10
4. Testing and Security Analysis 11
4.1 Vulnerability Detection Using Slither 11
4.2 Interpretation of Security Findings 11
4.3 Functional Testing 12
5. Deployment and DevOps 12
5.1 Testnet Deployment 12
5.2 Contract Verification 13
5.3 Frontend Deployment 13
5.4 CI/CD Pipeline 13
6. Frontend Implementation 14
6.1 Frontend Environment Configuration 14
6.2 Core Frontend Code Implementation 14
7. Discussion, Limitations, and Future Work 18
8. Conclusion (Conclusion and Discussion) 19

1. Introduction and Problem Analysis

1.1 Technical Background of Academic Incentives

Current academic incentive systems remain highly centralized, making them vulnerable to data tampering, loss, and operational inefficiencies. Reward distribution and auditing processes rely heavily on manual workflows, resulting in low transparency and limited scalability. Furthermore, student achievements are fragmented across multiple administrative systems, lacking a unified, verifiable, and persistent record. Existing literature highlights similar issues in traditional education data circulation platforms, where centralized architectures hinder trust, interoperability, and data integrity[i] (Education Circulation Platform Solution Based on Blockchain, n.d.).

1.2 Technical Adaptability of Blockchain

Blockchain technology provides core properties that directly address these structural limitations. Its immutability ensures that on-chain reward records are tamper-resistant and publicly verifiable, thus enhancing credibility and trust among stakeholders. Smart contract automation enables reward rules to be encoded as executable logic, supporting automatic reward distribution, self-service claims, and reduced administrative intervention. Blockchain systems also provide full auditability, where every reward action is transparently traceable, supporting accountable academic incentive mechanisms.

In addition, recent studies emphasize the value of NFTs for identity, achievement tracking, and digital credentialing, demonstrating their potential to serve as persistent, transferable proof of accomplishments in educational and non-educational contexts[ii] (A comprehensive study on Non-Fungible Tokens (NFTs), n.d.).

Recognizing that academic grades are high-frequency and variable data—making full on-chain storage economically impractical—this project adopts a hybrid design aligned with best practices: Off-chain Grade Management + On-chain Reward Storage and Distribution, leveraging blockchain where it provides meaningful value.

1.3 Project Technical Objectives (Lingnan CrownCred)

This project aims to build the Lingnan CrownCred (EduReward) prototype system to achieve the following objectives:

Off-chain Grade Management: Administrators input grades through the frontend interface, which are stored in JSON/local databases for reward eligibility computation

On-chain Reward Distribution: Students can receive ERC-20 token rewards and mint ERC-721 achievement NFTs based on their number of “A” grades. NFT metadata is stored through IPFS to ensure decentralization and persistence.

Closed-loop dApp Experience: The system provides integrated grade-management tools for administrators and viewing/claiming functionalities for students

Security and Verification: Static analysis is conducted using Slither, and the entire system is deployed on the Sepolia testnet for demonstration.

Furthermore, this incentive design is supported by empirical findings that cryptoeconomic mechanisms can positively influence human participation and knowledge-sharing behavior when properly structured[iii] (To incentivize or not, n.d.), reinforcing the project's foundational motivation.

2. System Design & Architecture

2.1 High-Level System Overview

Lingnan CrownCred employs a hybrid architecture of front-end dApp + Node.js + blockchain contract. However, in this project, we do not directly write grades onto the blockchain. Instead, we manage grade data in a more lightweight and controllable way. The overall system structure is as follows:

Front-end Interface (JavaScript)

- Administrators input student grades through the interface.
- Grades are saved to a back-end JSON file.
- Students log in with their usernames and read the JSON data through the front-end.
- If the grade meets the reward conditions (e.g., an A grade), the front-end calls the contract to issue tokens or mint NFTs.

Node.js Back-end

- Responsible for receiving grades input by the administrator.
- Writes and maintains the grade data in a local JSON file.
- Interacts with the smart contract via Ethers.js.

Blockchain Smart Contract (Solidity)

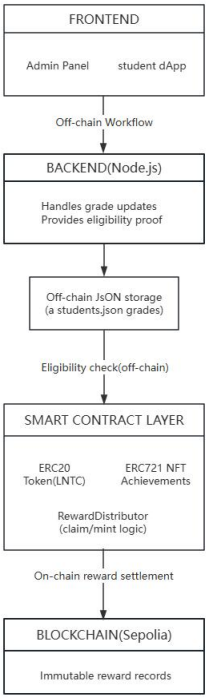
- Focuses on reward logic: issuing tokens, minting NFTs, and recording reward status.
- All reward-related statuses are transparent and immutable.

MetaMask Wallet

- The unique identity credential for students to receive rewards.
- Connects to the wallet via a browser; all rewards are initiated by the student themselves.

The core rationale behind this architecture is that performance data is high-frequency and variable, making on-chain storage costly and complex; while rewards are core assets, and on-chain storage better reflects their value and transparency.

Therefore, we adopt a structure of "off-chain performance data + on-chain rewards" to achieve a balance between cost, efficiency, and security.



2.2 Core Workflow Design

2.2.1 Grade Entry (Administrator Terminal)

This system does not include teacher-uploaded contracts; instead, it employs a simpler and more efficient method:

- Administrators enter student usernames and grades on the front end.
- The front end uses the Node.js API to write this record to a JSON file.
- The JSON file serves as the system's "off-chain database."
- Administrators can modify or update grades at any time.

Advantages include: In the prototype project, grades changed frequently and were unsuitable for on-chain storage; the JSON file structure is clear, traceable, and easy to debug; administrators do not need complex on-chain permission management, saving development and gas costs.

After a grade is successfully uploaded, the system will:

- Check if the number of A grades meets the NFT trigger conditions.
- Update the student's reward eligibility status.

a_students.json


```
{
  "a_students": [
    {
      "username": "20230001",
      "CourseId": "20250567"
    },
    {
      "username": "20230001",
      "CourseId": "20240112"
    },
    {
      "username": "20230002",
      "CourseId": "20250077"
    },
    {
      "username": "20240001",
      "CourseId": "20251022"
    },
    {
      "username": "20240002",
      "CourseId": "20250001"
    },
    {
      "username": "20240001",
      "CourseId": "20250001"
    },
    {
      "username": "20240003",
      "CourseId": "20250001"
    },
    {
      "username": "20230001",
      "CourseId": "20250001"
    }
  ]
}
```


2.2.2 Student Login and Eligibility Verification

After a student logs into their account using their username:

The frontend reads the student's grades from the JSON file and determines if they meet the reward criteria:

If they do:

- Display the corresponding Token/NFT available for claiming
- Provide a claim button

If they do not meet the criteria:

- Display the current grade status without showing the claim button

The key idea here is: Off-chain is responsible for determining eligibility, while on-chain is responsible for distributing rewards. This division of labor avoids security conflicts.。

2.2.3 NFT Minting Process

The smart contract will automatically unlock the corresponding level of NFT based on the student's accumulated A grades:

Number of A grades	Rewards NFT
1A	NFT1
2A	NFT2
3A	NFT3

The front-end will display the available NFTs in real-time on the student's NFT claim page.
NFT Minting Trigger Flow:
1. Front-end calls mintNFT()
2. MetaMask pops up a signature
3. Contract verifies eligibility → Mints NFT → Returns tokenId
4. Front-end updates the NFT display page
NFT metadata is stored via IPFS, ensuring decentralization and persistence.

2.2.4 Token Claiming Process

Students can connect to their MetaMask wallets via the front-end and then click "Claim Tokens".The process is as follows:
1. Front-end reads the current number of claimable tokens on the chain
2. Triggers the smart contract claimTokens()
3. Contract verifies whether the student meets the conditions (whether they have already claimed the tokens, whether their grades are satisfactory)
4. Transfers the fixed reward of 200 tokens to the student's address
5. Triggers the TokensClaimed event and updates the UI. All operations are signed by the student's own wallet; no teacher or administrator involvement is required.

2.2.5 Gift Redemption Process (Not Implemented)

Although the gift redemption process is not fully implemented, the expected design can be described in this section:

- The front end displays a list of gifts (name + required token)
- Students call the contract `exchangeGift(giftId)` through their wallets
- The contract deducts the student's token and generates a redemption record

2.3 Key Mechanism Design

Token & NFT Reward Rules are as follows:

Number of A grades	Token Rewards	NFT rewards	Can it be claimed repeatedly
1	200 Token	NFT1	No
2	200 Token	NFT2	No
3	200 Token	NFT3	No

3. Smart Contract Layer

The on-chain portion of Lingnan CrownCred consists of three types of smart contracts: experimental token LNTC (ERC-20), achievement badge NFTs (ERC-721), and a reward distribution contract.
Since performance evaluation is conducted off-chain, the smart contracts focus on "reward distribution, asset recording, and achievement verification," ensuring that once a student meets the reward criteria, the on-chain record is immutable and transparently verifiable.
The entire smart contract layer is designed according to three principles:
1. Dedicated Function: Grades are not stored on-chain; the on-chain system is only responsible for rewards.
2. Transparent and Auditable: All distribution actions are recorded in on-chain events.
3. Easy Front-End Calling: All distribution actions are completed through student wallet (MetaMask) signatures.

3.1 Design and Implementation of ERC-20 Token LNTC

The Lingnan CrownCred system creates an experimental token, **Lingnan Test Coin** (LNTC), based on the ERC-20 standard. This token is the on-chain underlying asset for the entire incentive mechanism.

3.1.1 Core Parameter Design

Before implementing the token, we first clarified its key parameters:

- Name：Lingnan Test Coin
- Symbol：LNTC
- Decimals：18
- Total Supply：A fixed initial quantity is used to ensure the "controllability of classroom experiments".
- Distribution Pool：The system reserves a portion of LNTC for student reward distribution.

These parameters ensure that LNTC complies with the ERC-20 standard specifications and also has the flexibility to expand reward mechanisms in the future.

3.1.2 ERC-20 Contract Construction and Testing

We base our implementation on the standard ERC-20 interface:

- Writing the contract
- Compiling the contract
- Deploying it on the test network (Sepolia)
- Verifying its functionality using a wallet and block explorer:
  - Transfer functionality
  - Balance inquiry
  - Allowance authorization mechanism
  - Compatibility (e.g., whether it can be correctly recognized by MetaMask)

These verifications ensure that LNTC fully complies with the ERC-20 ecosystem tool usage standards.

3.1.3 Incentive Pool Configuration

After deployment, we specifically allocate a portion of the total issuance as:

- Reward Pool

All reward tokens received by students come from this pool, and their distribution is strictly controlled by on-chain logic.

This design makes the reward distribution logic completely transparent and auditable on-chain.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.30;

import "https://github.com/OpenZeppelin/openzeppelin-contracts/blob/v4.9.3/contracts/token/ERC20/ERC20.sol";

contract MyGroupToken is ERC20 {
    constructor() ERC20("Lingnan test coin", "LNTC") {    infinite gas 904200 gas
        _mint(msg.sender, 10000000 * 10 ** decimals());
    }
}
```

3.2 Implementation and Metadata Storage of Achievement-Based NFTs (ERC-721)

To enhance the sense of ceremony and display value of students' academic achievements, we built an achievement badge NFT system, adhering to the ERC-721 standard.

3.2.1 NFT Image Material and Decentralized Storage (IPFS)

We used Pinata + IPFS to complete the following process:

1. Upload badge images for each achievement level to IPFS.
2. Obtain a decentralized content identifier (CID).

3. Manage the corresponding IPFS link as the NFT resource root.
This ensures that the NFT images are accessible from any node globally, without relying on a centralized server.

3.2.2 Metadata Creation

According to the ERC-721 Metadata standard, we create the following for each NFT:

- name (badge name)
- description (achievement description)
- image (IPFS image link)
- attributes (including attributes such as achievement level)

These metadata JSON files are then uploaded to IPFS, generating:

- Metadata URI

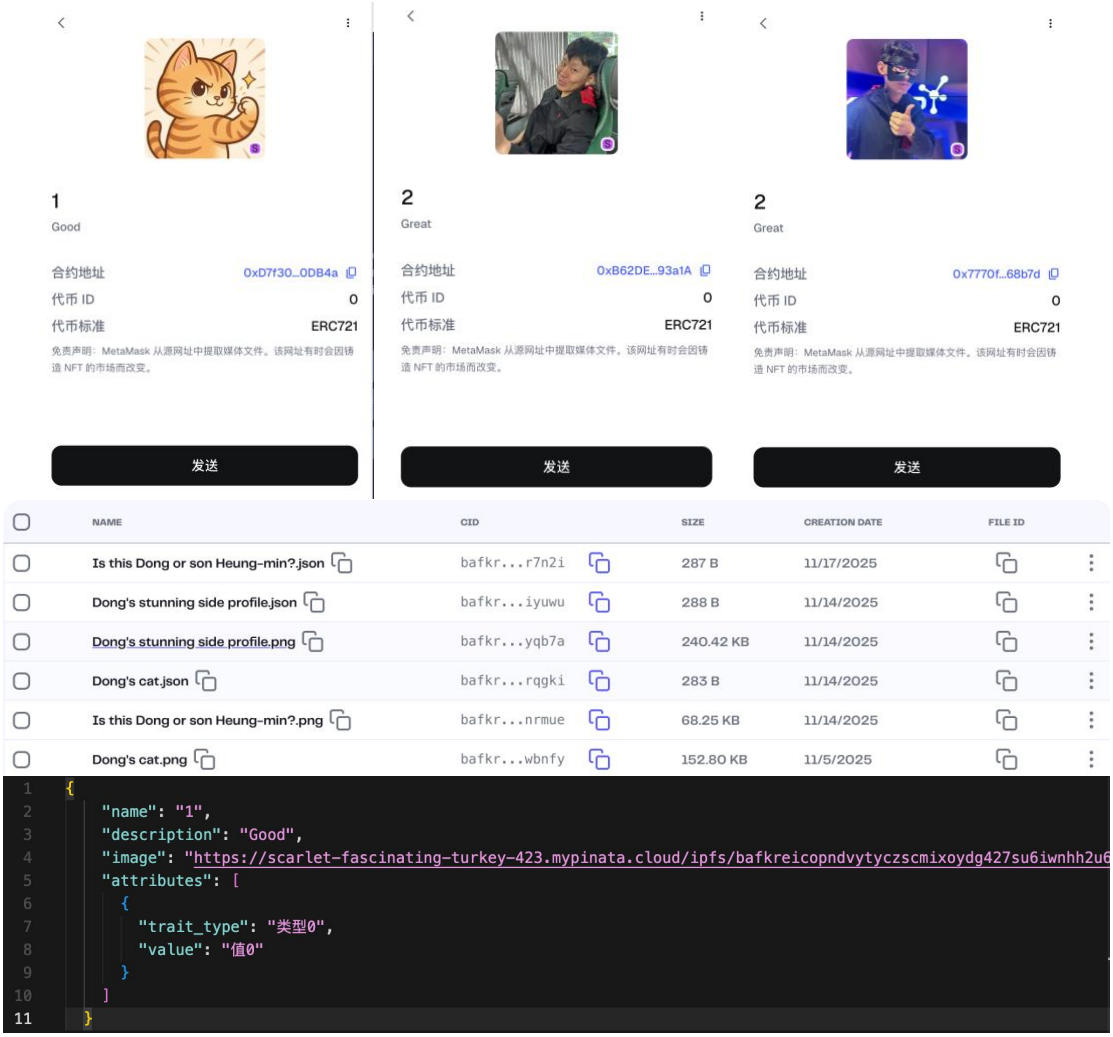
This URI is bound to a smart contract tokenId.

3.2.3 NFT Contract Deployment and TokenURI Mapping

The NFT contract implements:

- binding of tokenId to metadataURI
- uniqueness and immutability of NFTs
- minting events (NFTMinted) for front-end monitoring

Ultimately forming a displayable and traceable achievement badge system.



3.3 Automated Reward Distribution Contract

To improve reward distribution efficiency and ensure the on-chain reproducibility of reward actions, we designed a dedicated reward distribution contract for distributing:

- LNTC tokens
- Achievement-type NFTs

This contract collaborates with the token contract and the NFT contract to form the main logic for reward distribution.

3.3.1 Contractual Responsibilities: Unify on-chain distribution logic

Reward Distribution Contract Implementation:

- Automated Token Issuance (Unified Entry Point)
- Automated NFT Minting (Level-Based)
- Anti-Duplicate Claim Logic (On-Chain State Recording)
- Unified Event Output for Frontend Synchronization

This contract does not concern itself with performance scores; eligibility is determined by the frontend reading a JSON file.

It is only responsible for: issuing rewards according to the rules and recording the status as long as the frontend tells me I am "qualified".

3.3.2 Bulk airdrop mechanism (Airdrop)

To facilitate classroom demonstrations and one-time distribution, we designed:

- Batch distribution to multiple addresses
- Management of multiple students' collection lists
- Complete all distributions with a single function call

Significance:

- Significantly reduces operational costs
- Avoids the tediousness of individual on-chain transfers
- Enables more efficient function verification during demonstrations

This mechanism is a very common "operational-grade feature" in real Web3 projects.

3.3.3 Event Mechanism

Each reward action triggers an event, such as:

- Token Claimed(student, amount)
- NFT Minted(student, tokenId)
- Airdrop Executed(count)

The frontend listens for these events and updates the interface, displaying a message indicating successful claim.

Events also serve as:

- Transparent credentials
- Debugging tools
- Audit logs

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.30;

import "https://github.com/OpenZeppelin/openzeppelin-contracts/blob/v4.9.3/contracts/token/ERC20/IERC20.sol";
import "https://github.com/OpenZeppelin/openzeppelin-contracts/blob/v4.9.3/contracts/access/Ownable.sol";

contract TokenFaucet is Ownable {
    IERC20 public immutable token;
    uint256 public claimAmount;
    mapping(address => bool) public claimed;

    event Claimed(address indexed user, uint256 amount);
    event ClaimAmountUpdated(uint256 oldAmount, uint256 newAmount);
    event Withdrawn(address indexed to, uint256 amount);

    constructor(address tokenAddress, uint256 initialClaimAmount) Ownable() {    infinite gas 808600 gas
        require(tokenAddress != address(0), "token is zero");
        token = IERC20(tokenAddress);
        claimAmount = initialClaimAmount;
    }

    function claim() external {    infinite gas
        require(!claimed[msg.sender], "Already claimed");
        require(claimAmount > 0, "Claim amount is zero");
        require(
            token.balanceOf(address(this)) >= claimAmount,
            "Faucet has insufficient token balance"
        );

        claimed[msg.sender] = true;
```

```
        bool ok = token.transfer(msg.sender, claimAmount);
        require(ok, "Token transfer failed");

        emit Claimed(msg.sender, claimAmount);
    }

    function setClaimAmount(uint256 newAmount) external onlyOwner {    infinite gas
        uint256 old = claimAmount;
        claimAmount = newAmount;
        emit ClaimAmountUpdated(old, newAmount);
    }


    function withdraw(address to, uint256 amount) external onlyOwner {    infinite gas
        require(to != address(0), "to is zero");
        bool ok = token.transfer(to, amount);
        require(ok, "Withdraw failed");
        emit Withdrawn(to, amount);
    }
}
```

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.30;

import "https://github.com/OpenZeppelin/openzeppelin-contracts/blob/v4.9.3/contracts/token/ERC721/ERC721.sol";
import "https://github.com/OpenZeppelin/openzeppelin-contracts/blob/v4.9.3/contracts/access/Ownable.sol";

contract ExcellentStudentNFT1 is ERC721, Ownable {
    uint256 private _tokenIdCounter;
    mapping(address => bool) public hasMinted;

    string private constant TOKEN_URI =
        "https://scarlet-fascinating-turkey-423.mypinata.cloud/ipfs/bafkreiccu6aeefrx5wyuqrrx7ypfb7nq6y4pn425tu4bt57j

    constructor() ERC721(unicode"优秀学员NFT 1", "NFT1") Ownable() {}    infinite gas 2025400 gas

    function mint() external {    infinite gas
        require(!hasMinted[msg.sender], "Already minted");
        uint256 tokenId = _tokenIdCounter;
        _tokenIdCounter++;

        hasMinted[msg.sender] = true;
        _safeMint(msg.sender, tokenId);
    }

    function tokenURI(uint256 tokenId) public view override returns (string memory) {    infinite gas
        require(_exists(tokenId), "Nonexistent token");
        return TOKEN_URI;
    }
```

3.4 Interaction methods between the front-end and smart contracts

The entire reward process follows a pattern of off-chain eligibility assessment → on-chain reward execution → on-chain event feedback.

- Student login
- Frontend reads grades from a JSON file
- If eligible → Displays "Claim Reward"
- Student clicks to claim
- Frontend calls the distribution contract via MetaMask
- Distribution contract distributes LNTC or mints NFTs
- Event triggered → Frontend updates user reward status

This design ensures: flexible grades (modifiable off-chain), reliable rewards (immutable on-chain), a clear process, and traceability.

3.5 Smart Contract Layer Summary

Lingnan CrownCred's smart contract layer constitutes a clearly defined on-chain system:

Contract Type	Function	Is it on the chain?
ERC-20 LNTC	Incentive tokens	✔
ERC-721 Achievement NFT	Achievement Badges	✔
Reward Distribution Contract	Reward distribution, airdrops, and status recording	✔

This system achieves:

- Immutable on-chain rewards
- Smooth front-end interaction
- A closed-loop achievement system and token system

4. Testing and Security Analysis

This chapter conducts systematic testing and security analysis of the smart contracts of the Lingnan CrownCred system, focusing on static vulnerability detection, core security verification, and scenario-based functional testing to ensure the reliability and security of the overall system in the testnet environment.

4.1 Vulnerability Detection Using Slither

To systematically assess the risks of smart contracts, we configured a Solidity build version consistent with the project and installed Slither in a virtual environment. We then performed static analysis on all contracts involving core functions such as performance management, reward distribution, and NFT minting.

Slither's detection output presented all alerts, warnings, and their corresponding severity levels, primarily covering:

- Inconsistent compiler versions
- Potential security risks from external calls
- Unused or redundant functions
- Inheritance structure and import dependency issues

These findings provided a basis for further security diagnostics.

4.2 Interpretation of Security Findings

After a thorough analysis of Slither's detection results, we summarize the core issues as follows:

4.2.1 Reentrancy Attack Risk

The Slither report points out:

- There is a reentrancy point in `claim()`
- There is a reentrancy point in `withdraw()`
- Because the external call (token.transfer) occurs before the state change
- And the event is triggered after the external call

This is a typical case of an incorrect order of Checks-Effects-Interactions, which may be re-entered by `fallback()`.

```
Reentrancy in TokenFaucet.claim() (TokenFaucet.sol#27-41):
External calls:
- ok = token.transfer(msg.sender,claimAmount) (TokenFaucet.sol#37)
Event emitted after the call(s):
- Claimed(msg.sender,claimAmount) (TokenFaucet.sol#40)

Reentrancy in TokenFaucet.withdraw(address,uint256) (TokenFaucet.sol#51-56):
External calls:
- ok = token.transfer(to,amount) (TokenFaucet.sol#53)
Event emitted after the call(s):
- Withdrawn(to,amount) (TokenFaucet.sol#55)
```

4.2.2 Security of Token and NFT Issuance Logic

We focused on checking whether the functions related to token issuance and NFT minting followed the Checks–Effects–Interactions security pattern. It was verified that the contract modifies its state before making external calls, thus reducing the risk of reentrancy attacks.

4.2.3 Version and Dependency Consistency

The Slither report indicated that some contracts used inconsistent pragma declarations, which could lead to incompatible compilation behavior. We have upgraded all contracts to ^0.8.30 to ensure internal consistency and avoid known vulnerabilities.

```
6 different versions of Solidity are used:
- Version constraint ^0.8.20 is used by:
  - ^0.8.20 (node_modules/@openzeppelin/contracts/access/Ownable.sol#4)
  - ^0.8.20 (node_modules/@openzeppelin/contracts/token/ERC721/ERC721.sol#4)
- Version constraint ^0.8.30 is used by:
  - ^0.8.30 (nft3 (2).sol#2)
```

```
Version constraint ^0.8.20 contains known severe issues...
Version constraint >=0.8.4 contains known severe issues...
```

4.2.4 Redundant Code and Unused Functions

Slither identified some functions (such as the unrewritten `_baseURI()` and `_burn()`) that were not called. This redundancy did not affect system functionality but could potentially reduce maintainability; it has been removed or annotated during the cleanup process.
Overall, Slither's analysis provided clear directions for improving system security. After corrections, the system has no critical security vulnerabilities.

```
Context._contextSuffixLength() is never used and should be removed
ERC721._baseURI() is never used and should be removed
ERC721._burn(uint256) is never used and should be removed
```

4.3 Functional Testing

In addition to static analysis, we further conducted scenario-based functional testing based on the system's business logic to ensure the consistency and stability of on-chain and off-chain processes.
Functional testing covered the following core processes:
Student grade reading and verification

- Automatic NFT achievement badge minting
- LNTC token reward claiming and transfer verification
- Administrator-side grade entry and update process
- Front-end wallet connection and interaction with MetaMask

Each test case verified:

- Whether the on-chain state was correctly updated after the operation was executed
- Whether the front-end interface's feedback to the user was consistent with the on-chain results
- Whether there were issues such as duplicate claims, unupdated state, or incorrect calls.

5. Deployment and DevOps

This chapter introduces the deployment process, version management strategies, and DevOps automation mechanisms for smart contracts and front-end applications to ensure stable system operation and facilitate subsequent maintenance.

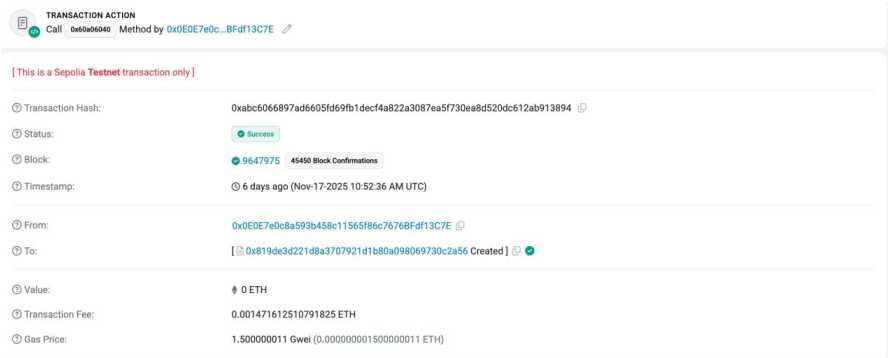
5.1 Testnet Deployment

The smart contract was compiled using the Remix tool and deployed to the Ethereum testnet: Sepolia.The following information was logged during the deployment process:

- ABI information came from the deployment tool Remix and had been configured in the frontend via ethers.js.

- Event Log and Transaction Hash
  - Token Contract Deployment:：
    - 0x9489c9Af20C2BBFA7c863B01778D172C64f0590D
  - NFT Contract Deployment:
    - 0xD7f30E42720c003bA8fA30C6Ba0b03cE47F0DB4a
    - 0xB62DE16d0F779e0305214b37bd5E350409993a1A
    - 0x7770f4Bc52dECc684FEC106db077c4D44c568b7d
  - Reward Claim Event: 0x819DE3d221D8a3707921d1b80A098069730c2a56
- Deployment Account
  - 0x0E0E7e0c8a593b458c11565f86c7676BFdf13C7E

After successful deployment, we verify the contract status in the block explorer. The front-end application then configures the contract address and ABI via environment variables, enabling the web interface to interact with the on-chain deployment instance.



5.2 Contract Verification

To enhance transparency and auditability, all smart contracts deployed on the testnet are submitted to a block explorer for verification. After contract verification:

- The source code is viewable by public users.
- The ABI is automatically published.
- Contract functions can be directly invoked via a browser.
- Third-party review and debugging are facilitated.

This step is a crucial component in ensuring project trustworthiness.

5.3 Frontend Deployment

The front-end is built and deployed to a hosting platform as a Node.js service. The application supports direct interaction with MetaMask, enabling functions such as on-chain reward claiming and NFT display.

5.4 CI/CD Pipeline

The system uses Git to manage source code and leverages continuous integration processes for automated builds. The process includes:

1. Code push triggers automated testing
2. Static smart contract analysis (Slither)
3. Frontend build and compression
4. Automated deployment to a hosting platform. This overall process ensures that updated versions are automatically deployed after verification, reducing manual deployment errors and improving project iteration efficiency.

6. Frontend Implementation

6.1 Frontend Environment Configuration

6.1.1 Technical Stack Selection

Based on the project's core architecture of "off-chain performance management + on-chain reward distribution," the front-end technology stack focuses on compatibility, ease of blockchain interaction, and development efficiency. Specific selections are as follows:

- The project uses Next.js 13 (Pages Router) paired with TypeScript as the core framework. SSR is used to optimize first-screen loading and SEO, while TypeScript ensures code type safety and reduces blockchain interaction errors.
- The Chakra UI component library is used to quickly implement a "deep blue + gold" brand visual through theme customization. Its component-based design improves development efficiency and adapts to scenarios such as wallet connection and reward display.
- Blockchain interaction combines Ethers.js v5 (responsible for underlying contract interaction) and Wagmi v2. The latter simplifies repetitive logic such as MetaMask connections and contract calls through Hooks-based APIs.
- React Query is used to cache on-chain data (such as reward status), effectively reducing duplicate requests; it also automatically handles request status (loading, success, failure), optimizing the user experience.

6.1.2 Environment Setup Process

1. Project Initialization: Build a basic Next.js + TypeScript project using create-next-app, and configure development tools such as ESLint and PostCSS.
2. Core Dependency Installation: Install Chakra UI component libraries, Wagmi/Ethers.js blockchain tools, and the React Query data caching library.
3. Global Configuration Initialization is completed in _app.tsx: It sets the Chakra UI theme color scheme, initializes Wagmi and binds it to the Sepolia testnet with automatic reconnection, injects the React Query client, and configures the default request retry and caching mechanisms.
4. Development Environment Debugging: Configure the Next.js development server port (default 3000), resolve dependency version conflicts (such as React 18 adaptation, Wagmi v2 module path adjustments), and ensure that blockchain interaction-related dependencies load correctly.

6.1.3 Configuration Verification

Verify environment availability by creating a test page: Theme verification was completed, confirming that the Chakra UI components correctly display the dark blue (#1E40AF) and gold (#D97706) color scheme; the blockchain connection test successfully initialized the Wagmi client and recognized MetaMask in the browser; dependency compatibility checks showed no missing modules, and contract calls to related APIs can be imported and used normally.

6.2 Core Frontend Code Implementation

6.2.1 Project Structure

A modular directory structure is used to ensure clear functional responsibilities:

```
score-chain-frontend/
│
├── 📄 配置文件
│   ├── package.json            # 项目依赖配置
│   ├── tsconfig.json           # TypeScript配置
│   ├── next.config.ts          # Next.js配置
│   └── eslint.config.mjs       # ESLint配置
│
├── 📱 页面层 (pages/)
│   ├── _app.tsx                # 全局应用配置
│   ├── _document.tsx           # HTML文档结构
│   ├── index.tsx               # 首页
│   ├── score-entry.tsx         # 成绩录入中心
│   ├── profile.tsx              # 个人中心
│   ├── shop.tsx                 # 兑换商城
│   ├── nft-mint.tsx            # NFT铸造
│   ├── claim.tsx                # 领取奖励
│   ├── admin.tsx               # 管理员面板
│   └── api/                    # API路由
│       ├── a_students/write.ts
│       ├── wallet/address.ts
│       └── hello.ts
│
├── 🧩 组件层 (components/)
│   ├── Navbar.tsx              # 导航栏
│   ├── LoginModal.tsx          # 登录弹窗
│   ├── walletButton.tsx        # 钱包连接按钮
│   ├── BackButton.tsx          # 返回按钮
│   ├── ClaimButton.tsx         # 领取奖励按钮
│   ├── NFTMintButton.tsx       # NFT铸造按钮
│   ├── NetworkChecker.tsx      # 网络检查器
│   ├── NFTProductCard.tsx     # NFT商品卡片
│   │
│   ├── access/                 # 权限控制组件
│   │   ├── FeatureGuard.tsx    # 功能访问保护
│   │   └── PermissionGate.tsx  # 钱包权限门禁
│   │
│   ├── score-entry/            # 成绩录入组件
│   │   ├── SingleEntryForm.tsx # 单条录入表单
│   │   ├── BatchUploadPanel.tsx # 批量导入面板
│   │   ├── EntryHistory.tsx    # 操作历史
│   │   └── ScoreQuery.tsx      # 成绩查询
│   │
│   ├── score-entry.tsx        # 📝 功能页面1: 成绩录入中心
│   ├── profile.tsx            # 👤 功能页面2: 个人中心
│   ├── shop.tsx               # 🛒 功能页面3: 兑换商城
│   ├── nft-mint.tsx           # 🎨 功能页面4: NFT铸造
│   └── claim.tsx              # 💰 功能页面5: 领取奖励
│
└── 🧩 components/             # 组件层
    └── LoginModal.tsx         # 🔐 登录界面
```

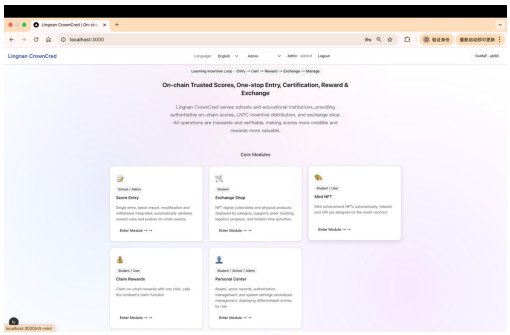
6.2.2 Key Page Implementation

1) Main Page (index.tsx)

Functionality: The platform's homepage, displaying navigation cards for core modules and a function overview.

This front-end page obtains translated text through an internationalization context, dynamically generates an array of cards containing five functional modules, and uses SimpleGrid for a responsive layout. Each card links to its corresponding functional page.

Screenshots:

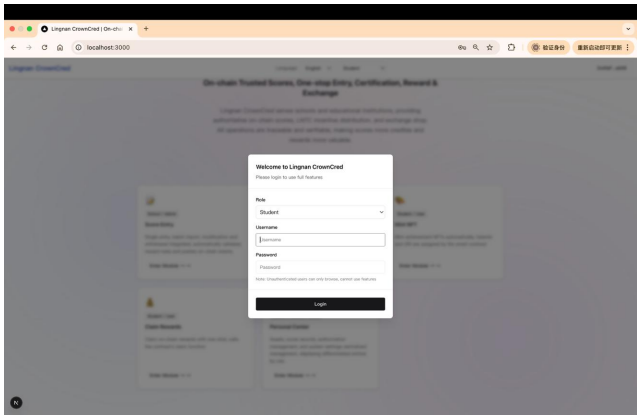


2) Login Interface

Functionality: Forced login pop-up, supports administrator/student role login verification.

This core logic includes a fixed full-screen overlay (which cannot be closed). When the form is submitted, the parent component's onLogin function is called, and it supports switching between three roles: admin, student, and third. At the same time, error messages and loading status are displayed in real time.

Screenshots:



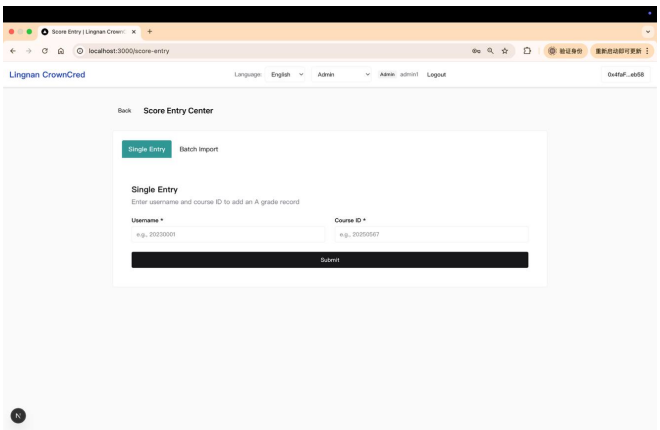
3) Function Page 1: Grade Entry Center (score-entry.tsx)

Function: Administrators can enter or batch import A grade records in JSON format.

Core Logic:

The core logic restricts access to administrators only through FeatureGuard and uses a tab to switch between two modes: single entry (via SingleEntryForm) and batch import (via BatchUploadPanel to process JSON files).

Screenshots:

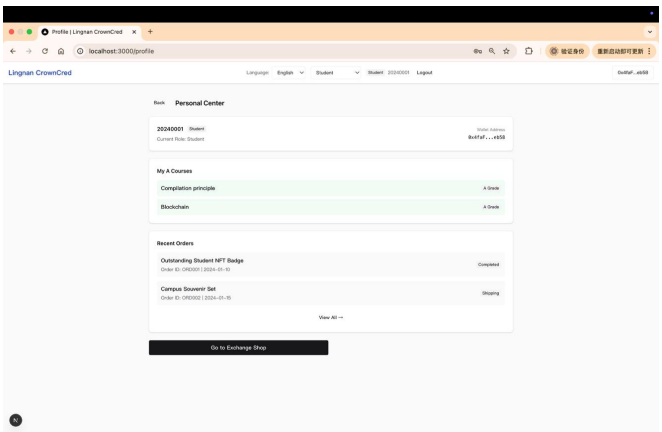


4) Function Page 2: Personal Center (profile.tsx)

Function: Displays user information, a list of Course A, reward records, and order records.

Core logic is: Retrieve user role information from AuthContext; retrieve the student's list of Course A from a JSON file in real time; retrieve reward records using RewardContext; and listen for the aStudentsUpdated event to update Course A information in real time.

Screenshot:

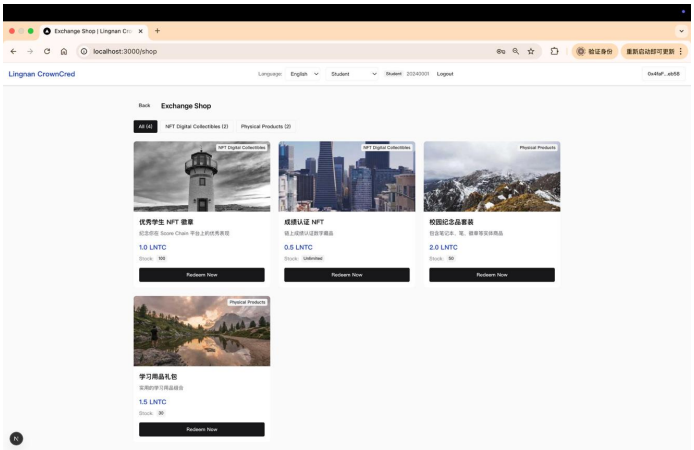


5) Function Page 3: Redemption Mall (shop.tsx)

Function: Displays NFT digital collectibles and physical goods, supporting category filtering and redemption.

The core logic is as follows: Use ShopContext to obtain the product list; switch between All/NFT/Physical Product categories via Tab; administrators can add new products (via AddProductForm); product cards support redemption by calling smart contracts.

Screenshot:

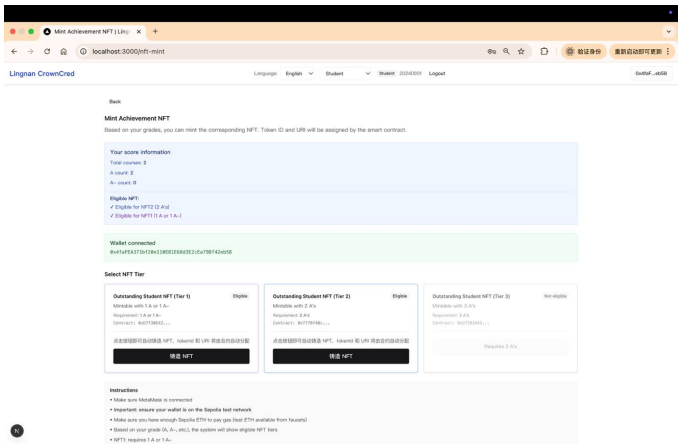


6) Function Page 4: NFT Minting (nft-mint.tsx)

Function: Mint achievement NFTs corresponding to the student's grade A record.

The core logic is: retrieve the A course record from a_students.json, calculate the number of A grades, check NFT minting eligibility (NFT1/NFT2/NFT3), then display the mintable NFT levels, and call the corresponding contract's mint() function to perform the minting.

Screenshot:

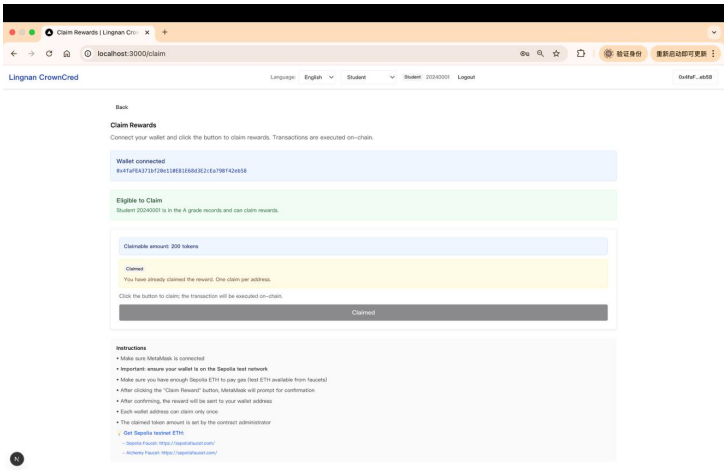


7) Function Page 5: Claim Rewards (claim.tsx)

Function: One-click claiming of preset on-chain token rewards; each address can only claim once.

Core logic: Checks if the student qualifies for the reward as shown in a_students.json, then calls the smart contract's claim() function via the ClaimButton component; the contract automatically checks and prevents duplicate claims, and displays the current claim status and contract information.

Screenshot:



7. Discussion, Limitations, and Future Work

Lingnan CrownCred employs a hybrid architecture of "off-chain grades + on-chain rewards," significantly reducing the cost and complexity of putting high-frequency grades on-chain while ensuring transparent and traceable rewards. However, as a course prototype system, it still has the following limitations and areas for improvement:

Off-chain grades still rely on centralized trust.
Currently, grades are entered off-chain by administrators and stored in JSON/localStorage. The system's trust in the authenticity of grades still comes from teachers/administrators. Future considerations could include:
Introducing grade hashes or Merkle Roots on-chain to provide a "verifiable but not plaintext" proof of grade integrity; introducing multi-signature/school node verification mechanisms or trusted oracles to reduce single-point trust issues.

The incentive rules are relatively static.
Currently, the reward mechanism is a fixed threshold and a fixed number of tokens (reaching the target of 200 tokens + tiered NFTs), which is easy to understand but has limited adaptability to different courses/semesters. Future expansion could include: dynamically adjusting rewards based on course difficulty and credit weight; adding multi-dimensional incentives such as ranking and continuous progress to form a richer academic achievement system.

The redemption mall is currently only a prototype.
While the mall display and redemption process are running smoothly, inventory, orders, and logistics are still primarily maintained off-chain. Future improvements could include: on-chain inventory/order management and event-driven settlement processes; support for more product types, NFT redemption rights, and binding with real-world discounts/certificates.

Scalability and cross-school recognition are not yet implemented.
Currently, the system is designed for single-course/single-school scenarios. Future expansion could extend to multiple courses, multiple colleges, and even cross-school recognition, forming a more universal academic credit certificate based on NFT achievements.

8. Conclusion (Conclusion and Discussion)

The Lingnan CrownCred project demonstrates a feasible path for introducing blockchain into educational incentive scenarios. By keeping high-frequency, variable performance data off-chain and recording low-frequency, high-value rewards and achievements on-chain, the system reduces gas costs and management complexity while still achieving transparency, auditability, and immutability in the reward distribution process.

At the implementation level, the project has completed an on-chain closed loop for ERC-20 incentive tokens and ERC-721 achievement NFTs, and supports administrators' performance entry, student eligibility viewing, token redemption, and tiered NFT minting through a front-end dApp; it has also been deployed and demonstrated on the Sepolia testnet. Static auditing and key scenario testing by Slither demonstrate that the core contract logic possesses basic security and operability, validating the overall design of "flexible off-chain management + trustworthy on-chain incentives."

Work Discussion and Project Reflection

The development of this project mainly revolved around three main lines: "off-chain performance management—on-chain reward distribution—front-end closed-loop interaction." The biggest challenge during implementation was ensuring consistency between the stated goals and the actual implementation. Initially, we envisioned storing results on-chain, but after assessing costs, privacy, and variability, we adjusted to off-chain storage. This required simultaneous adjustments to contract responsibilities, front-end data flow, and reporting narratives to ensure a closed-loop system logic.

Another key challenge was the stability and user experience of the front-end Web3 interaction, such as wallet connection state management, transaction waiting and failure rollback, and UI synchronization after event listening. These all required separating the presentation layer from the business logic layer to reduce coupling and improve maintainability.

From a project management perspective, we broke down the work into modules (results/rewards/contracts/front-end), connecting each part through phased integration testing and a unified interface (Context + Hooks). This approach effectively reduced the conflict costs of multi-person collaboration and made subsequent expansion smoother. Overall, Lingnan CrownCred not only achieved its prototype goals but also provided a clear engineering foundation and evolutionary direction for the subsequent introduction of a more reliable results input mechanism, dynamic incentive rules, and a complete on-chain exchange economy.

---

[i] *Education Circulation Platform Solution Based on Blockchain (n.d.) [Whitepaper/Report].*

[ii] *A comprehensive study on Non-Fungible Tokens (NFTs): Use cases, ecosystem, benefits & challenges (n.d.) [Academic Article].*

[iii] *To incentivize or not: Impact of blockchain-based cryptoeconomic tokens on human information sharing behavior (n.d.) [Research Article.*